

July 30, 2026

```
[1]: # Numpy für bessere Berechnungen
import numpy as np
from numpy import exp, sqrt, log, pi
from uncertainties import unumpy as unp

# Weiteres für bessere Rechnungen
import pylab as py

from decimal import Decimal, ROUND_CEILING, ROUND_HALF_UP, getcontext #
    ↳ Besonders für sig. Runden
import math

# Berechnungen und Plotting
from scipy import odr
import scipy.optimize
import scipy.constants as cc
from scipy.stats import norm
from scipy.optimize import curve_fit
from scipy.stats import chi2
from scipy.stats import poisson
from scipy.integrate import quad
from scipy.signal import find_peaks
from scipy.signal import argrelextrema, argrelmin, argrelmax
from scipy.special import factorial
import scipy.integrate as integrate
from scipy.special import gamma

%matplotlib widget
import matplotlib.pyplot as plt
from matplotlib import colormaps
import matplotlib.mlab as mlab
import matplotlib.transforms as transforms

# Zum Auslesen von Dateien und ähnlichem
import os
import os.path
from pathlib import Path
```

```

import pandas as pd # Auch wichtig für den Latex Export
from pytexit import py2tex
import csv
import re
from typing import List

# Besseres Funktionen handling
import sympy as sp
from sympy import separatevars

# Display und Output
from IPython.display import display, Math, Latex, HTML

import textwrap

# Grafiken in EU-Größen ausgeben
def figsize_cm(width_cm, height_cm):
    return (2*width_cm / 2.54, 2*height_cm / 2.54)
def comma_to_float(valstr):
    return float(valstr.replace(',', '.'))

g=9.80984
Delta_g=0.00002

```

```

[2]: currentversuch = "252"
base = Path.cwd()
Messwerte_path = Path(base.parent.parent.parent/"Messwerte"/currentversuch)
    ↪ # initializing paths
Ausgabe_path = Path(base/"Diagramme")
print(Messwerte_path)
print(Ausgabe_path)

```

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2\Messwerte\252

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2\Auswertungen2.2\Code\252\Diagramme

Delete UTF8 False encoding

```

[3]: # def deletevocals(text):
#     text=textwrap.dedent(text)
#     patterna = r'["][a] '
#     text=re.sub(patterna, "ä", text)
#     patternu=r'["][u] '
#     text=re.sub(patternu, "ü", text)
#     patterno=r'["][o] '
#     text=re.sub(patterno, "ö", text)
#     return text

```

```

# Regexpwissen: [0-9], [A-Z] findet egal welchen Zahl/Buchstaben; [^0-9] findet
↳ erstes Zeichen, das keine Zahl ist;

def deletevocal(text):
    # Einrückungen entfernen (gemeinsame Tabs des gesamten Textes)
    text = textwrap.dedent(text)
    # entfernt Bindestriche: - findet Bindestrich, \n Zeilenumbruch direkt
↳ dahinter, \s Whitespaces *beliebig viele
    text = re.sub(r'-\n\s*', "", text)
    # entfernt Zeilenumbrüche
    text = text.replace("\n", " ")

    # Eine Funktion, die vom re.sub aufgerufen wird, wenn ein Treffer erzielt
↳ wird
    def convert(match):
        # match.group(1) ist der Buchstabe nach dem Pünktchen (z.B. 'a' oder
↳ 'A')
        buchstabe = match.group(1)
        # Dictionary für die echten Umlaute
        umlaute = {'a': 'ä', 'u': 'ü', 'o': 'ö', 'A': 'Ä', 'U': 'Ü', 'O': 'Ö'}
        # Gib den Umlaut zurück. Falls das Zeichen nicht im Dict ist, lass es
↳ wie es ist
        return umlaute.get(buchstabe, buchstabe)
    # Das Pattern sucht nach ` gefolgt von einem beliebigen Buchstaben aus der
↳ Auswahl, [X] findet genau ein Zeichen aus Menge X
    return re.sub(r'`([a-zA-Z])', convert, text)

```

```

[4]: print(deletevocal("""
"""))

```

Runden signifikanter Stellen

```

[5]: def round_to_sigs(val, errVal=None, einheiten=None):
    """
    Funktion zur Rundung eines Fehlers und die Anpassung des Messwertes daran.
↳ Diese Funktion wurde etwas umständlicher
    geschrieben, da python mit Floats und Runden schnell in Probleme rennt.
↳ Daher musste hier mit dezimal gearbeitet werden.
    Zudem sollten besonders kleine und große Messwerte in der
↳ Dezimalschreibweise geschrieben werden, damit diese auch
    für Protokolle geeignet sind.

    Parameter

```

```

-----
**val** : float
    Messwert

**errVal** : float
    Ungenauigkeit des Messwertes

Return
-----
**value_rounded** : str
    Gerundeter Messwert

**error_rounded** : str
    Gerundete Ungenauigkeit des Messwertes

**res** : str
    "Messwert \pm Fehler"
"""
einheiten_str = einheiten if einheiten is not None else ""
if errVal is not None:
    # Daten zu Dezimal wechseln, da Floats probleme machen
    val = Decimal(str(val))
    errVal = Decimal(str(errVal))

    exp = int(math.floor(math.log10(float(errVal)))) # Die erste
    ↳signifikante Stellenposition wird anhand des errVal bestimmt

    if round(float(errVal / (Decimal(10) ** exp))) < 3: # wenn
    ↳1,2,3 erste signifikante Stelle, dann kommt eine zweite
    ↳Nachkommastellenposition hinzu
        exp -= 1

    scale = Decimal(10) ** exp

    val_round = (val / scale).quantize(Decimal('1'),
    ↳rounding=ROUND_HALF_UP) * scale # mit scale wird das Komma so
    ↳verschoben, dass alle signifikanten Stellen vor dem Komma stehen, und dann
    ↳alle Nachkommastellen weggerundet werden
    err_round = (errVal / scale).quantize(Decimal('1'),
    ↳rounding=ROUND_CEILING) * scale

    num = f"\num{{{val_round}}({err_round})}"
    qty = f"\qty{{{val_round}}({err_round})}{{{einheiten_str}}}" if
    ↳einheiten else num

    return float(val_round), float(err_round), num, qty
else:

```

```

    val = Decimal(str(val))
    exp = int(math.floor(math.log10(float(val))))
    if round(float(val / (Decimal(10) ** exp))) < 3:           # wenn 1,2,3
↳erste signifikante Stelle, dann kommt eine zweite Nachkommastellenposition
↳hinzu
        exp -= 1
    scale = Decimal(10) ** exp
    val_round = (val / scale).quantize(Decimal('1'),
↳rounding=ROUND_HALF_UP) * scale
    num = f"\\num{{{val_round}}}"
    qty = f"\\qty{{{val_round}}}{einheiten_str}" if einheiten else num
    return float(val_round), None, num, qty

v_round = np.vectorize(round_to_sigs, otypes=[float, float, object, object],
↳excluded=['einheiten'])

    # wissenschaftliche Notation erstmal vernachlässigen (da häufig Einheiten
↳gewollt sind, die 10^e Präfixe beinhalten)

```

Gauss'sche Fehlerfortpflanzung

```

[6]: def gff(function, errPronePar, latex=True):
    """
    Kann die Fehlerformel einer gegebenen Gleichung bestimmen.

    Parameters
    -----
    **func** : sympy function
        Funktion dessen Fehler bestimmt werden soll.

    **errPronePar** : Array
        Liste (Array) aller fehlerbehafteten Größen der Gleichung con sp.Symbols
        Diese Werte werden als x_sym, y_sym, z_sym etc. bezeichnet und sind
↳ungleich den Werten für x, y, z.
        Für die Werte wird daher die Bezeichnung x_val, y_val, z_val etc.
↳genutzt und für deren Fehler err_x, err_y, err_z etc.

    Return
    -----
    **absolut_err** : sympy function
        Gibt die Fehlergleichung des absoluten Fehlers wieder.

    **relativ_err** : sympy function
        Gibt die Fehlergleichung des relativen Fehlers wieder.

    **errProneParameters** : array
        Liste aller Fehlerbehafteten Größen

```

```

"""

error = 0
errProneParameters = []

function = sp.simplify(function)
variables = errPronePar.split(",")
variables = sp.symbols(variables)

for variable in variables:
    Delta = sp.Symbol(f"Delta_{variable}")
    partial = sp.diff(function, variable) # Die Funktion
    → wird nach der fehlerbehafteten Variable abgeleitet
    term = sp.UnevaluatedExpr(partial*Delta)**2
    error = error + ((term)) # Fehler werden
    → quadratisch aufsummiert
    errProneParameters.append((variable, Delta))

absolute_err = sp.simplify(sp.sqrt(error), rational = True)
relative_err = sp.simplify(sp.sqrt(error/function**2), rational = True)

if latex:
    # Prints out the Latex Code for the Functions
    print("Funktion:")
    display(Math(sp.latex(function, long_frac_ratio=2)))
    print(sp.latex(function))
    print("Absoluter Fehler:")
    display(Math(sp.latex(absolute_err, long_frac_ratio=2)))
    print(sp.latex(absolute_err))
    print("Relativer Fehler:")
    display(Math(sp.latex(relative_err, long_frac_ratio=2)))
    print(sp.latex(relative_err))
    return function, absolute_err, relative_err, errProneParameters

```

Sigma-Abweichungen

```

[7]: def sigma_abweichung(p1, err_p1, p2, err_p2):
    """
    Funktion zum berechnen der Sigma-Abweichung von zwei Messwerten, oder einem
    → Messwert und einem Literaturwert.
    """
    if err_p1 == 0 and err_p2 == 0:
        raise ValueError("Für die Sigma-Abweichung muss mindestens ein Wert
        → fehlerbehaftet sein!")
    else:
        abweichung = Decimal(str(abs(p1 - p2)/(np.sqrt(err_p1 ** 2 + err_p2 ** 2))))

```

```

if abweichung == 0:
    return 0.0, "\\num{0}"

exp = int(math.floor(math.log10(float(abweichung))))
if round(float(abweichung / (Decimal(10) ** exp))) < 3:           # wenn
    ↪ 1,2,3 erste signifikante Stelle, dann kommt eine zweite
    ↪ Nachkommastellenposition hinzu
    exp -= 1
scale = Decimal(10) ** exp
abweichung_round = (abweichung / scale).quantize(Decimal('1'),
    ↪ rounding=ROUND_CEILING) * scale
res = f"\\num{{{abweichung_round}}}"

return float(abweichung_round), res

```

```

[8]: # #Größenvergleich in latex
# def size_comp_str(name1,name2,val1,val2):
#     if val1<val2:
#         return name1+"<"+name2
#     elif val1>val2:
#         return name1+">"+name2
#     else:
#         return name1+"="+name2

#Erstellung von Vergleichstabellen in Latex
def compare_table_array(nam1, nam2, einheiten, val1_array, err1_array,
    ↪ val2_array, err2_array):
    # 1. Tabellenkopf (Hier werden die Strings einmalig eingesetzt)
    output_header = textwrap.dedent(f"""
        \\begin{{table}}[htbp]
        \\centering
        \\caption{{Vergleich von ${nam1}$ und ${nam2}$}}
        \\begin{{tabularx}}{{0.8\\textwidth}}{{ZZZZZ}}
        \\toprule
        Messreihe & {nam1}[\\unit{{{einheiten}}}] &
    ↪ {nam2}[\\unit{{{einheiten}}}] & \\text{{abs.Abw.}}[\\unit{{{einheiten}}}] &
    ↪ \\text{{proz.Abw.}}[\\unit{{\\percent}}] & S[\\sigma] \\\\midrule
        """).strip() # strip macht whitespaces am anfang und ende des strings weg

    table_rows = []

    # 2. Der Body (Schleife läuft über die Werte-Arrays)
    for val1, err1, val2, err2 in zip(val1_array, err1_array, val2_array,
    ↪ err2_array):

        VAL1=round_to_sigs(val1,err1,r'[2]
        if err2!=0:

```

```

        VAL2=round_to_sigs(val2,err2,r'')[2]
    else:
        VAL2=val2

    abs=np.abs(val1-val2)
    abs_delta=np.sqrt(err1**2+err2**2)
    if abs_delta!=0:
        SIGMA=sigma_abweichung(val1,err1,val2,err2)[1]
    else:
        SIGMA=0.0

    rel=abs/np.abs(val1)*100
    if abs != 0:
        rel_delta = rel * np.sqrt((abs_delta / abs)**2 + (err1 / val1)**2)
        ABS = round_to_sigs(abs, abs_delta, r'')[2]
        REL = round_to_sigs(rel, rel_delta, r'')[2]
    else:
        rel_delta = 0
        ABS = "0"
        REL = "0"

    # Eine saubere Zeile nur mit den Werten für LaTeX
    row = f"          & {VAL1} & {VAL2} & {ABS} & {REL} & {SIGMA} \\\\"
    table_rows.append(row)

body_str = "\n".join(table_rows)

# 3. Tabellenfuß
output_footer = textwrap.dedent(f"""
    \\bottomrule
    \\end{{tabularx}}
    \\label{{table:Vergleich_{{nam1}}}}
    \\end{{table}}
""").strip()

# Alles zusammensetzen
final_output = f"{output_header}\\n{body_str}\\n{output_footer}"

print(final_output)

```

```

[9]: # #Erstellung von Vergleichstabellen in Latex
# def compare_table_withliterature(nam1,nam2,einheiten,val1,err1,val2):
#     VAL1=round_to_sigs(val1,err1,r'')[2]
#     abs=np.abs(val1-val2)
#     abs_delta=np.sqrt(err1**2)
#     if abs_delta!=0:
#         SIGMA=sigma_abweichung(val1,err1,val2,0)[1]

```

```

else:
#         SIGMA=0.0
#         ABS=round_to_sigs(abs,abs_delta,r'')[2]
#         rel=abs/np.abs(val2)*100
#         rel_delta=rel*np.sqrt((abs_delta/abs)**2+(err1/val1)**2) if abs != 0 else 0
#     REL=round_to_sigs(rel,rel_delta,r'')[2]
#     output = textwrap.dedent(f"""
#         \begin{table}[htb]
#         \centering
#         \caption{{}}
#         \begin{tabularx}{{0.8\textwidth}}{{ZZZZZ}}
#         \toprule
#         \begin{array}{ccccc}
#             \text{{\nam1}} & \text{{\unit{{einheiten}}}} & \text{{\nam2}} & \text{{\unit{{einheiten}}}} & \text{{abs.Abw.}} \\
#             & \text{{\unit{{einheiten}}}} & & \text{{proz.Abw.}} & \\
#             & \text{{\unit{{percent}}}} & \text{{\sigma}} & \text{{\midrule}} & \\
#             \text{{VAL1}} & \text{{\num{{val2}}}} & \text{{ABS}} & \text{{REL}} & \text{{SIGMA}} \\
#             & \text{{\bottomrule}} & & & \\
#             & \text{{\end{tabularx}}} & & & \\
#             & \text{{\label{table:Vergleich_{{nam1}}}}} & & & \\
#             & \text{{\end{table}}} & & & \\
#         """)
#     print(output)

# #Größenvergleich in latex
# def size_comp_str(name1,name2,val1,val2):
#     if val1<val2:
#         return name1+"<"+name2
#     elif val1>val2:
#         return name1+">"+name2
#     else:
#         return name1+"="+name2

#Erstellung von Vergleichstabellen in Latex
import textwrap
def compare_table(nam1,nam2,einheiten,val1,err1,val2,err2):
    VAL1=round_to_sigs(val1,err1,r'')[2]
    if err2!=0:
        VAL2=round_to_sigs(val2,err2,r'')[2]
    else:
        VAL2=val2
    abs=np.abs(val1-val2)
    abs_delta=np.sqrt(err1**2+err2**2)
    if abs_delta!=0:

```

```

        SIGMA=sigma_abweichung(val1,err1,val2,err2)[1]
    else:
        SIGMA=0.0

    rel=abs/np.abs(val1)*100
    if abs != 0:
        rel_delta = rel * np.sqrt((abs_delta / abs)**2 + (err1 / val1)**2)
        ABS = round_to_sigs(abs, abs_delta, r')[2]
        REL = round_to_sigs(rel, rel_delta, r')[2]
    else:
        rel_delta = 0
        ABS = "0"
        REL = "0"

    output = textwrap.dedent(f"""
        \\begin{{table}}[htbp]
        \\centering
        \\caption{{Vergleich von ${nam1}$ und ${nam2}$}}
        \\begin{{tabularx}}{{\\textwidth}}{{ZZZx}}
        \\toprule
        \\
        ↪{{nam1}}[\\unit{{einheiten}}]&{{nam2}}[\\unit{{einheiten}}]&\\text{{abs.Abw.
        ↪}}[\\unit{{einheiten}}]&\\text{{proz.Abw.
        ↪}}[\\unit{{percent}}]&S[\\sigma]\\\\\\midrule
        {VAL1}&{VAL2}&{ABS}&{REL}&{SIGMA}\\\\\\
        \\bottomrule
        \\end{{tabularx}}
        \\label{{table:Vergleich_{nam1}}}
        \\end{{table}}
    """)
    print(output)

```

```

[35]: N=[]
torzeit=10
for i in range(1,5):
    pfad=f"silber{i}.txt"
    n=np.loadtxt(Messwerte_path/"Silber"/pfad,
    ↪delimiter=",",usecols=[1],skiprows=4,unpack=True)
    N.append(n)
n=np.sum(N,axis=0)/(4*torzeit)
Delta_n=np.sqrt(np.sum(N,axis=0))/(4*torzeit)

relFehler0=np.mean(Delta_n/n)
relFehler1=1/2*np.mean(1/np.sqrt(n*torzeit))
relFehler2=1/2*1/sqrt(torzeit)*np.mean(1/np.sqrt(n))
print("relFehler:deltan/n",relFehler0)
print("relFehler:sqrt(N)",relFehler1)

```

```

print("relFehler:sqrt(T)",relFehler2)

t=np.arange(5,400,10)
background=np.loadtxt(Messwerte_path/"Silber"/"untergrund.txt",
    ↪delimiter=",",skiprows=4,usecols=[1])
mean_background=np.mean(background)/torzeit
Delta_background=np.std(background/torzeit)/np.sqrt(len(background)-1)
print("background",mean_background)

def fit_func(x, A1,l1,A2,l2):
    return A1*np.exp(-x*l1) + A2*np.exp(-x*l2) + y_0
y_0array=mean_background+np.array([-Delta_background,Delta_background,0])
plotlabel=[r"Fit mit $n_0-\Delta n_0$",r"Fit mit $n_0-\Delta n_0$",r"Fit mit $n_0$",
    ↪"$n_0$",]
plt.close('all')
fig, ax1 = plt.subplots(1, 1, figsize=(figsize_cm(14,6)))
# fig.suptitle(r'Silbermessung')
ax1.errorbar(t,n,yerr=Delta_n,fmt='.',capsize= 3,elinewidth= 0.
    ↪5,label='Messwerte')# ecolour='black'
ax1.set_title(r'Zerfall von  $^{108}\mathrm{Ag}$  und  $^{110}\mathrm{Ag}$ ')
ax1.set_xlabel(r'Zeit $t$ [s]')
ax1.set_ylabel(r'Zählrate $n$; [\text{Bq}]$')
ax1.set_yscale('log')
# ax1.axhline(mean_background,linestyle=(0, (5,
    ↪3)),color='black',label=r'Untergrund $n_0\pm\Delta n_0$')
# ax1.axhline(mean_background-Delta_background,linewidth=0.6,color='black',)
# ax1.axhline(mean_background+Delta_background,linewidth=0.6,color='black',)

lambda1=[]
lambda2=[]
Delta_lambda1=[]
Delta_lambda2=[]
A1=[]
A2=[]
Delta_A1=[]
Delta_A2=[]

for i in range(3):
    y_0=y_0array[i]
    popt, pcov=curve_fit(fit_func,t[2:],n[2:], p0=[500,0.02,50,0.
    ↪002],sigma=Delta_n[2:], absolute_sigma=True)
    if i==2:
        poptspeicher=popt
    ax1.plot(t,fit_func(t,*popt),label=plotlabel[i])
    lambda1.append(popt[1])
    Delta_lambda1.append(np.sqrt(pcov[1][1]))
    lambda2.append(popt[3])

```

```

Delta_lambda2.append(np.sqrt(pcov[3][3]))

A1.append(popt[0])
Delta_A1.append(np.sqrt(pcov[0][0]))
A2.append(popt[2])
Delta_A2.append(np.sqrt(pcov[2][2]))
print(poptspeicher)
print("lambda1=",lambda1)
print("Delta_lambda1=",Delta_lambda1)
print("lambda2=",lambda2)
print("Delta_lambda2=",Delta_lambda2)

Delta_l1=np.sqrt(np.mean([np.abs(lambda1[2]-lambda1[0]),np.
↪abs(lambda1[2]-lambda1[1])])**2+Delta_lambda1[2]**2)
l1,Delta_l1,_,latexl1=round_to_sigs(lambda1[2],Delta_l1,r'\per\s')
print("l1=",latexl1)
Delta_l2=np.sqrt(np.mean([np.abs(lambda2[2]-lambda2[0]),np.
↪abs(lambda2[2]-lambda2[1])])**2+Delta_lambda2[2]**2)
l2,Delta_l2,_,latexl2=round_to_sigs(lambda2[2],Delta_l2,r'\per\s')
print("l2=",latexl2)

DELTA_A1=np.sqrt(np.mean([np.abs(A1[2]-A1[0]),np.
↪abs(A1[2]-A1[1])])**2+Delta_A1[2]**2)
_,_,_,latexA1=round_to_sigs(A1[2],DELTA_A1,r'\per\s')
print("A1=",latexA1)
DELTA_A2=np.sqrt(np.mean([np.abs(A2[2]-A2[0]),np.
↪abs(A2[2]-A2[1])])**2+Delta_A2[2]**2)
_,_,_,latexA2=round_to_sigs(A2[2],DELTA_A2,r'\per\s')
print("A2=",latexA2)

T_1=np.log(2)/l1
T_2=np.log(2)/l2
Delta_T_1=np.log(2)*Delta_l1/l1**2
Delta_T_2=np.log(2)*Delta_l2/l2**2
T_1,Delta_T_1,_,latexT1=round_to_sigs(T_1,Delta_T_1,r'\s')
T_2,Delta_T_2,_,latexT2=round_to_sigs(T_2,Delta_T_2,r'\s')
print("T1=",latexT1)
print("T2=",latexT2)

tau1=1/l1
Delta_tau1=Delta_l1/l1**2
tau1,Delta_tau1,_,latextau1=round_to_sigs(tau1,Delta_tau1,r'\s')
print("tau1=",latextau1)
tau2=2/l2
Delta_tau2=Delta_l2/l2**2
tau2,Delta_tau2,_,latextau2=round_to_sigs(tau2,Delta_tau2,r'\s')
print("tau2=",latextau2)

```

```

chi2_f=np.sum((fit_func(t[2:],*poptspeicher)-n[2:]）**2/Delta_n[2:]**2)
dof_f=len(n[2:])-4 #dof:degrees of freedom, Freiheitsgrad
chi2_red=chi2_f/dof_f
print("chi2=", round_to_sigs(chi2_f)[0])
print("chi2_red=",round_to_sigs(chi2_red)[0])
prob=round(1-chi2.cdf(chi2_f,dof_f),2)*100
print("Wahrscheinlichkeit=", prob,"%")

ax1.legend()
# ax1.grid()

plt.tight_layout()
# plt.savefig(Ausgabe_path/'silberlog.png',format="png",bbox_inches='tight')
plt.show()

# m,Delta_m,_,latexm=round_to_sigs(popt[0],np.
#   ↳sqrt(pcov[0][0]),r'\volt\per\hertz')
# print("links:")
# print(f"m={latexm}")
# print("c="+round_to_sigs(popt[1],np.sqrt(pcov[1][1]),r'\milli\volt')[3])
# print("rechts:")
# print(r"m="+round_to_sigs(poptI[1],np.
#   ↳sqrt(pcovI[0][0]),r'\milli\volt\per\ampere')[3])
# print(r"c="+round_to_sigs(poptI[0],np.sqrt(pcovI[1][1]),r'\milli\volt')[3])

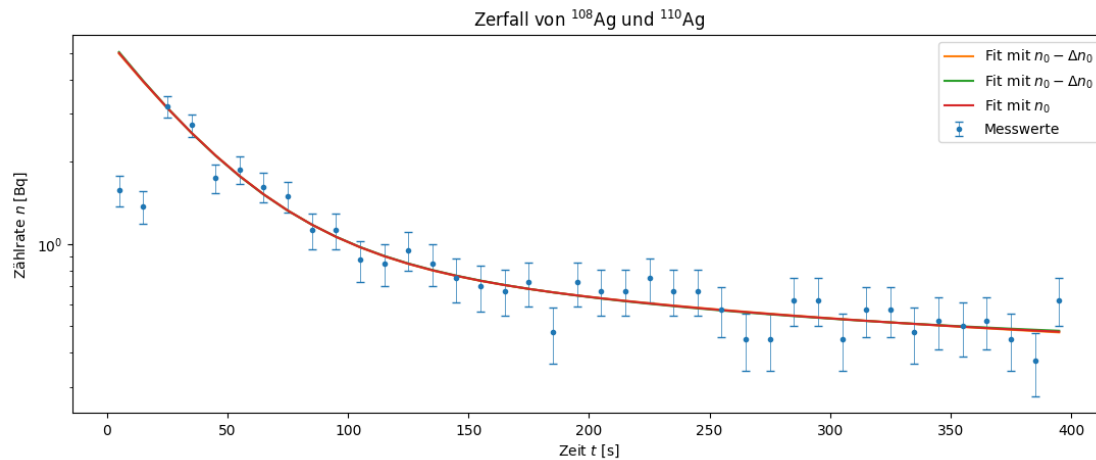
```

```

relFehler:deltan/n) 0.18346787481425433
relFehler:sqrt(N) 0.18346787481425433
relFehler:sqrt(T) 0.18346787481425436
background 0.38125
[4.63648739 0.02990991 0.65241358 0.00483349]
lambda1= [np.float64(0.029379146937190723), np.float64(0.03073100866797658),
np.float64(0.029909914269914094)]
Delta_lambda1= [np.float64(0.007035389142916432),
np.float64(0.008852347110782243), np.float64(0.007713908799220168)]
lambda2= [np.float64(0.00408575179595493), np.float64(0.005891853523965589),
np.float64(0.004833485902578989)]
Delta_lambda2= [np.float64(0.0018668422450091556),
np.float64(0.0026884436966492344), np.float64(0.0022041577033376837)]
l1= \qty{0.030(0.008)}{\per\s}
l2= \qty{0.0048(0.0024)}{\per\s}
A1= \qty{4.6(0.9)}{\per\s}
A2= \qty{0.7(0.4)}{\per\s}
T1= \qty{23(7)}{\s}
T2= \qty{140(80)}{\s}
tau1= \qty{33(9)}{\s}
tau2= \qty{420(110)}{\s}

```

chi2= 19.0
 chi2_red= 0.6
 Wahrscheinlichkeit= 98.0 %



```
[34]: torzeit=120
N=np.loadtxt(Messwerte_path/"indium.txt",
             ↪delimiter=" ",usecols=[1],skiprows=4,unpack=True)
n=N/torzeit
Delta_n=np.sqrt(N)/torzeit
t=np.arange(5,3000,120)
relFehler0=np.mean(Delta_n/n)
relFehler1=np.mean(1/np.sqrt(N))
relFehler2=1/sqrt(torzeit)*np.mean(1/np.sqrt(n))
print("relFehler:deltan/n",relFehler0)
print("relFehler:sqrt(N)",relFehler1)
print("relFehler:sqrt(T)",relFehler2)

background=np.loadtxt(Messwerte_path/"untergrund_indium.txt",
                      ↪delimiter=" ",skiprows=4,usecols=[1])
mean_background=np.mean(background)/10
Delta_background=np.std(background/10)/np.sqrt(len(background)-1)
print("background",mean_background)

def fit_func(x, A1,l1):
    return A1*np.exp(-x*l1) + y_0
y_0array=mean_background+np.array([-Delta_background,Delta_background,0])
plotlabel=[r"Fit mit $n_0-\Delta n_0$",r"Fit mit $n_0-\Delta n_0$",r"Fit mit_
           ↪$n_0$",]

plt.close('all')
```

```

fig, ax1 = plt.subplots(1, 1, figsize=(figsize_cm(14,6)))
# fig.suptitle(r'Silbermessung')
ax1.errorbar(t,n,yerr=Delta_n,fmt='.',capsize= 3,elinewidth= 0.
    ↳5,label='Messwerte')# ecolor='black'
ax1.set_title(r'Zerfall von  $^{116}\mathrm{In}$ ')
ax1.set_xlabel(r'Zeit  $t$  [s]')
ax1.set_ylabel(r'Zählrate  $n$  [Bq]')
ax1.set_yscale('log')
# ax1.axhline(mean_background,linestyle=(0, (5,↳
    ↳3)),color='black',label=r'Untergrund  $n_0 \pm \Delta n_0$ ')
# ax1.axhline(mean_background-Delta_background,linewidth=0.6,color='black',)
# ax1.axhline(mean_background+Delta_background,linewidth=0.6,color='black',)

lambda1=[]
Delta_lambda1=[]
for i in range(3):
    y_0=y_0array[i]
    popt, pcov=curve_fit(fit_func,t,n, p0=[500,0.02],sigma=Delta_n,↳
    ↳absolute_sigma=True)
    if i==2:
        poptspeicher=popt
        ax1.plot(t,fit_func(t,*popt),label=plotlabel[i])
        lambda1.append(popt[1])
        Delta_lambda1.append(np.sqrt(pcov[1][1]))
Delta_l1=np.sqrt(np.mean([np.abs(lambda1[2]-lambda1[0]),np.
    ↳abs(lambda1[2]-lambda1[1]))**2+Delta_lambda1[2]**2)
l1,Delta_l1,_,latexl1=round_to_sigs(lambda1[2],Delta_l1,r'\per{s}')
print("l1=",latexl1)
DELTA_A1=np.sqrt(np.mean([np.abs(A1[2]-A1[0]),np.
    ↳abs(A1[2]-A1[1]))**2+Delta_A1[2]**2)
_,_,_,latexA1=round_to_sigs(A1[2],DELTA_A1,r'\per{s}')
print("A1=",latexA1)

T_1=np.log(2)/l1
Delta_T_1=np.log(2)*Delta_l1/l1**2
T_1,Delta_T_1,_,latexT1=round_to_sigs(T_1,Delta_T_1,r'\s')
print("T1",latexT1)

tau=1/l1
Delta_tau=Delta_l1/l1**2
tau,Delta_tau,_,latextau=round_to_sigs(tau,Delta_tau,r'\s')
print("tau",latextau)

chi2_f=np.sum((fit_func(t[2:],*poptspeicher)-n[2:]）**2/Delta_n[2:]**2)
dof_f=len(n[2:])-2 #dof:degrees of freedom, Freiheitsgrad
chi2_red=chi2_f/dof_f

```

```

print("chi2=", round_to_sigs(chi2_f)[0])
print("chi2_red=", round_to_sigs(chi2_red)[0])
prob=round(1-chi2.cdf(chi2_f,dof_f),2)*100
print("Wahrscheinlichkeit=", prob,"%")

ax1.legend()
# ax1.grid()

plt.tight_layout()
# plt.savefig(Ausgabe_path/'indiumlog.png',format="png",bbox_inches='tight')
plt.show()

# m,Delta_m,_,latexm=round_to_sigs(popt[0],np.
#   ↪sqrt(pcov[0][0]),r'\volt\per\hertz')
# print("links:")
# print(f"m={latexm}")
# print("c="+round_to_sigs(popt[1],np.sqrt(pcov[1][1]),r'\milli\volt')[3])
# print("rechts:")
# print(r"m="+round_to_sigs(poptI[1],np.
#   ↪sqrt(pcovI[0][0]),r'\milli\volt\per\ampere')[3])
# print(r"c="+round_to_sigs(poptI[0],np.sqrt(pcovI[1][1]),r'\milli\volt')[3])

```

relFehler:deltan/n) 0.04622544934289126

relFehler:sqrt(N) 0.04622544934289126

relFehler:sqrt(T) 0.046225449342891245

background 0.33333333333333337

$l_1 = \sqrt[4]{0.000218(0.000012)} \{ \text{per} \text{s} \}$

$A_1 = \sqrt[4]{4.6(0.9)} \{ \text{per} \text{s} \}$

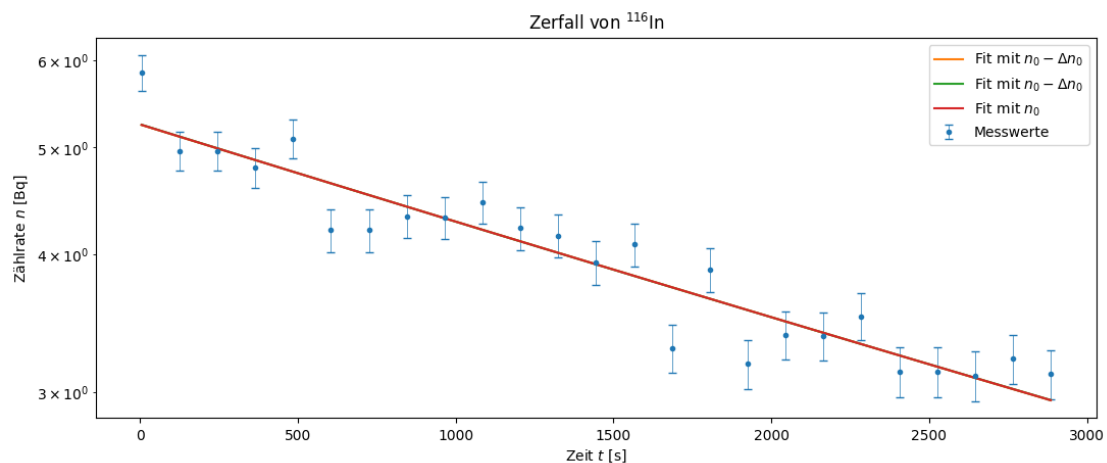
$T_1 \sqrt[4]{3180(180)} \{ \text{s} \}$

$\tau \sqrt[4]{4600(300)} \{ \text{s} \}$

chi2= 30.0

chi2_red= 1.7

Wahrscheinlichkeit= 3.0 %



```

[33]: N=[]
torzeit=10
for i in range(1,5):
    pfad=f"silber{i}.txt"
    n=np.loadtxt(Messwerte_path/"Silber"/pfad,
        ↪delimiter=",",usecols=[1],skiprows=4,unpack=True)
    N.append(n)
n=np.sum(N,axis=0)/(4*torzeit)
Delta_n=np.sqrt(np.sum(N,axis=0))/(4*torzeit)

t=np.arange(5,400,10)
background=np.loadtxt(Messwerte_path/"Silber"/"untergrund.txt",
    ↪delimiter=",",skiprows=4,usecols=[1])
mean_background=np.mean(background)/torzeit
Delta_background=np.std(background/torzeit)/np.sqrt(len(background)-1)
print("background",mean_background)

def fit_func(x, A1,l1,A2,l2):
    return A1*np.exp(-x*l1) + A2*np.exp(-x*l2) + y_0
y_0array=mean_background+np.array([-Delta_background,Delta_background,0])
plotlabel=[r"Fit mit $n_0-\Delta n_0$",r"Fit mit $n_0-\Delta n_0$",r"Fit mit
    ↪$n_0$",]

plt.close('all')
fig, (ax1,ax2) = plt.subplots(2, 1, sharey=True,figsize=(figsize_cm(14,12)))
# fig.suptitle(r'Silbermessung')
ax1.errorbar(t,n,yerr=Delta_n,fmt='.',capsize= 3,elinewidth= 0.
    ↪5,label='Messwerte')# ecolore='black'
ax1.set_title(r'Zerfall von  $^{108}\mathrm{Ag}$  und  $^{110}\mathrm{Ag}$ ')
ax1.set_xlabel(r'Zeit $t$ [s]')
ax1.set_ylabel(r'Zählrate $n$; [\text{Bq}]$')
ax1.set_yscale('log')
ax1.axhline(mean_background,linestyle=(0, (5,
    ↪3)),color='black',label=r'Untergrund $n_0\pm\Delta n_0$')
ax1.axhline(mean_background-Delta_background,linewidth=0.6,color='black',)
ax1.axhline(mean_background+Delta_background,linewidth=0.6,color='black',)

lambda1=[]
lambda2=[]
Delta_lambda1=[]
Delta_lambda2=[]
for i in range(3):
    y_0=y_0array[i]

```

```

    popt, pcov=curve_fit(fit_func,t[2:],n[2:], p0=[500,0.02,50,0.
↳002],sigma=Delta_n[2:], absolute_sigma=True)
    if i==2:
        poptspeicher=popt
        ax1.plot(t,fit_func(t,*popt),label=plotlabel[i])
        lambda1.append(popt[1])
        Delta_lambda1.append(np.sqrt(pcov[1][1]))
        lambda2.append(popt[3])
        Delta_lambda2.append(np.sqrt(pcov[3][3]))

ax1.legend()
# ax1.grid()

torzeit=120
N=np.loadtxt(Messwerte_path/"indium.txt",
↳delimiter=",",usecols=[1],skiprows=4,unpack=True)
n=N/torzeit
Delta_n=np.sqrt(N)/torzeit
t=np.arange(5,3000,120)
relFehler0=np.mean(Delta_n/n)
relFehler1=np.mean(1/np.sqrt(N))
relFehler2=1/sqrt(torzeit)*np.mean(1/np.sqrt(n))
print("relFehler:deltan/n",relFehler0)
print("relFehler:sqrt(N)",relFehler1)
print("relFehler:sqrt(T)",relFehler2)

background=np.loadtxt(Messwerte_path/"untergrund_indium.txt",
↳delimiter=",",skiprows=4,usecols=[1])
mean_background=np.mean(background)/10
Delta_background=np.std(background/10)/np.sqrt(len(background)-1)
print("background",mean_background)

def fit_func(x, A1,l1):
    return A1*np.exp(-x*l1) + + y_0
y_0array=mean_background+np.array([-Delta_background,Delta_background,0])

ax2.errorbar(t,n,yerr=Delta_n,fmt='.',capsize= 3,elinewidth= 0.
↳5,label='Messwerte')# ecolor='black'
ax2.set_title(r'Zerfall von  $^{116}\mathrm{In}$ ')
ax2.set_xlabel(r'Zeit  $t$  [s]')
ax2.set_ylabel(r'Zählrate  $n$  [Bq]')
ax2.set_yscale('log')
ax2.axhline(mean_background,linestyle=(0, (5,
↳3)),color='black',label=r'Untergrund  $n_0 \pm \Delta n_0$ ')
ax2.axhline(mean_background-Delta_background,linewidth=0.6,color='black',)
ax2.axhline(mean_background+Delta_background,linewidth=0.6,color='black',)

```

```

lambda1=[]
Delta_lambda1=[]
for i in range(3):
    y_0=y_0array[i]
    popt, pcov=curve_fit(fit_func,t,n, p0=[500,0.02],sigma=Delta_n,
↳absolute_sigma=True)
    if i==2:
        poptspeicher=popt
    ax2.plot(t,fit_func(t,*popt),label=plotlabel[i])
    lambda1.append(popt[1])
    Delta_lambda1.append(np.sqrt(pcov[1][1]))

ax2.legend()

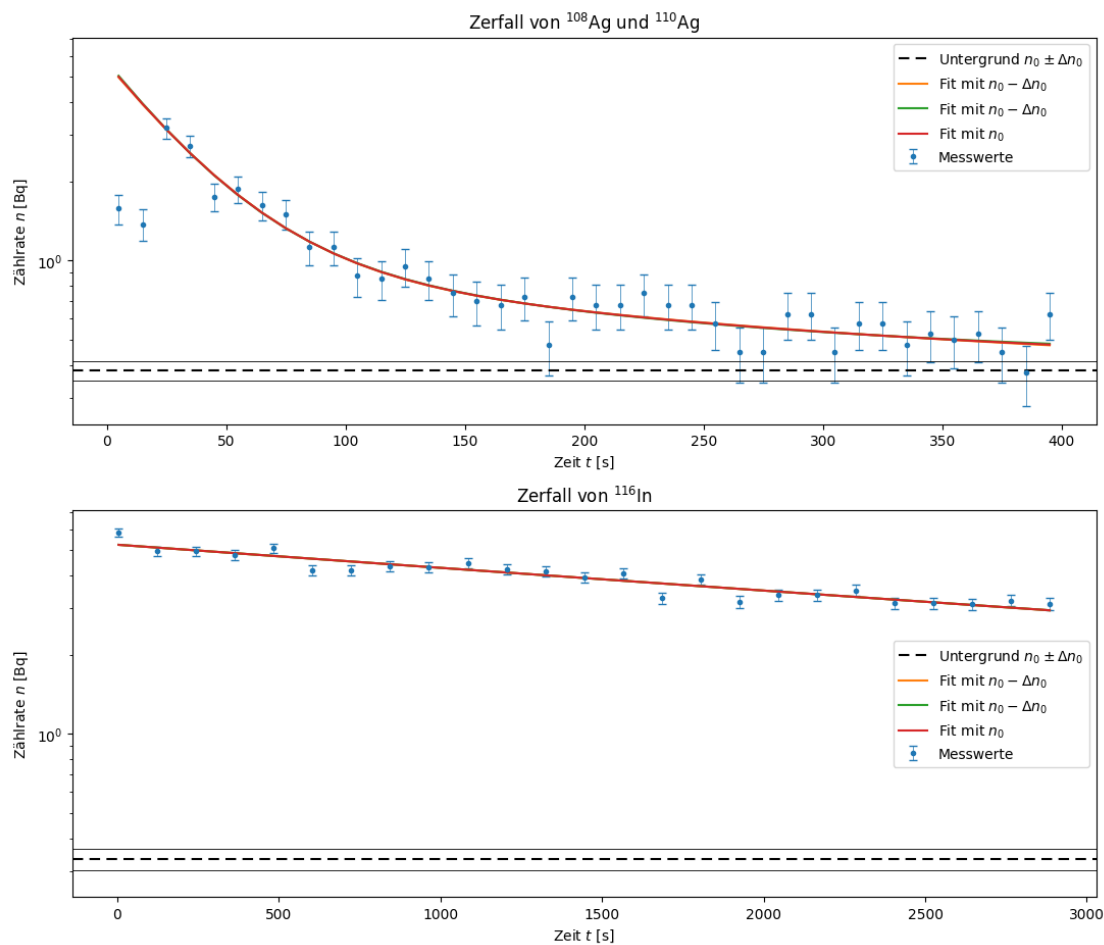
plt.tight_layout()
plt.savefig(Ausgabe_path/'silber_indiumlog.
↳png',format="png",bbox_inches='tight')
plt.show()

```

```

background 0.38125
relFehler:deltan/n) 0.04622544934289126
relFehler:sqrt(N) 0.04622544934289126
relFehler:sqrt(T) 0.046225449342891245
background 0.33333333333333337

```



```
[13]: compare_table_array(r'T_{\nicefrac{1}{2},\text{exp}}',r'T_{\nicefrac{1}{2},\text{Lit}}',r'\s',
    ↪41*60,24.6],[0,0])
```

```
\begin{table}[htbp]
\centering
\caption{Vergleich von  $T_{\frac{1}{2},\text{exp}}$  und
 $T_{\frac{1}{2},\text{Lit}}$ }
\begin{tabularx}{0.8\textwidth}{ZZZZZ}
\toprule
Messreihe &  $T_{\frac{1}{2},\text{exp}}$ [\unit{s}] &
 $T_{\frac{1}{2},\text{Lit}}$ [\unit{s}] & \text{abs.Abw.}[\unit{s}] &
\text{proz.Abw.}[\unit{percent}] &  $S[\sigma]$  \\
\midrule
& \num{140(80)} & 144.60000000000002 & \num{0(80)} & \num{0(60)} &
\num{0.06} \\
& \num{23(7)} & 24.6 & \num{2(7)} & \num{10(40)} & \num{0.23} \\
\bottomrule
\end{tabularx}
\label{table:Vergleich_T_{\frac{1}{2},\text{exp}}}
```

```
\end{table}
```

```
[14]: compare_table(r'T_{\nicefrac{1}{2}},\text{exp}',r'T_{\nicefrac{1}{2}},\text{Lit}',r's',3180,1
↪14*60,0)
```

```
\begin{table}[htbp]
\centering
\caption{Vergleich von  $T_{\frac{1}{2}}$ ,  $\text{exp}$  und
 $T_{\frac{1}{2}}$ ,  $\text{Lit}$ }
\begin{tabularx}{\textwidth}{ZZZx}
\toprule
T_{\frac{1}{2}}, \text{exp} [\unit{s}] & T_{\frac{1}{2}}, \text{Lit} [
\unit{s}] & \text{abs. Abw.} [\unit{s}] & \text{proz. Abw.} [\unit{percent}] & S[\text{sign}
a] \\ \midrule
\text{num}{3180(180)} & \text{num}{3248.4} & \text{num}{70(180)} & \text{num}{2(6)} & \text{num}{0.4} \\
\bottomrule
\end{tabularx}
\label{table:Vergleich_T_{\frac{1}{2}}, \text{exp}}
\end{table}
```

```
[15]: gff("a/b", "a, b")
```

Funktion:

$$\frac{a}{b}$$

```
\frac{a}{b}
```

Absoluter Fehler:

$$\sqrt{\frac{1}{b^4} (\Delta_a^2 b^2 + \Delta_b^2 a^2)}$$

```
\sqrt{\frac{\Delta_a^2 b^2 + \Delta_b^2 a^2}{b^4}}
```

Relativer Fehler:

$$\sqrt{\frac{\Delta_a^2}{a^2} + \frac{\Delta_b^2}{b^2}}$$

```
\sqrt{\frac{\Delta_a^2}{a^2} + \frac{\Delta_b^2}{b^2}}
```

```
[15]: (a/b,
sqrt((Delta_a**2*b**2 + Delta_b**2*a**2)/b**4),
sqrt(Delta_a**2/a**2 + Delta_b**2/b**2),
[(a, Delta_a), (b, Delta_b)])
```

```
[ ]: A_0_110=4.6 # 110
A_0_108=0.7 # 108
Delta_A_0_110=0.9
Delta_A_0_108=0.4
```

```

T_12=np.array([24.6,144.6]) # 110, 108

VA=A_0_110/A_0_108 # 110/108
Delta_VA=VA*np.sqrt((Delta_A_0_110/A_0_110)**2+(Delta_A_0_108/A_0_108)**2)
print(VA,Delta_VA)
VT=T_12[1]/T_12[0] #108/110 da A_0\sim 1/T
print(VA,VT)
t=420
V_A=(93.4*49*(1-np.exp(-np.log(2)/T_12[0]*t)))/(34.8*51*(1-np.exp(-np.log(2)/
↪T_12[1]*t))) # 110/108
print(V_A)

```

```

6.571428571428571 3.969112314036067
6.571428571428571 5.878048780487804
5.749999999999999
2.9760904695730965

```